



[Signalsmith Audio](#) / [Blog](#) / [Designing a straightforward limiter](#)

Designing a straightforward limiter

Geraint Luff

Signalsmith Audio Ltd.

2022-04-08

We set out some requirements for a brick-wall limiter, have a go at solving them, and sketch out some ways it could be modified.

[Introduction](#)

[Varying gain over time](#)

[Smoothing out the gain changes](#)

[Release curve](#)

[Extra hold time](#)

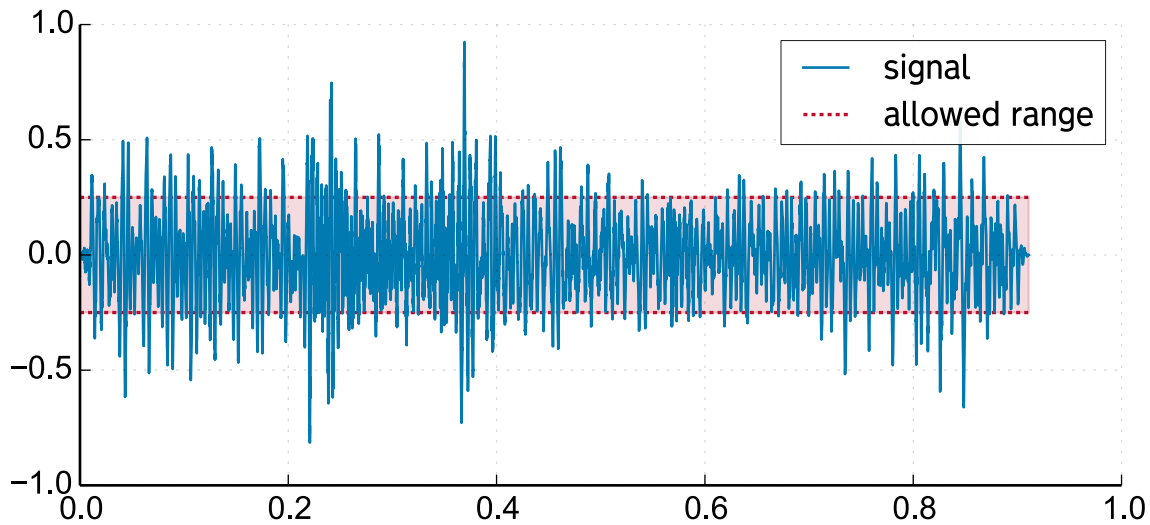
[Examples](#)

[Alternative release curves](#)

[... and even more choices](#)

[Conclusion](#)

So, here's the challenge. We have some input signal, but it goes outside the "allowed" range we have set:



Our allowed range here is between ± 0.25 , or approximately -12 dB.

0:00 / 0:12

We want to modify this signal such that it stays within the specified range. We want to preserve the loudness to some degree, and we *don't* want to reduce the gain in quieter sections where we don't need to.

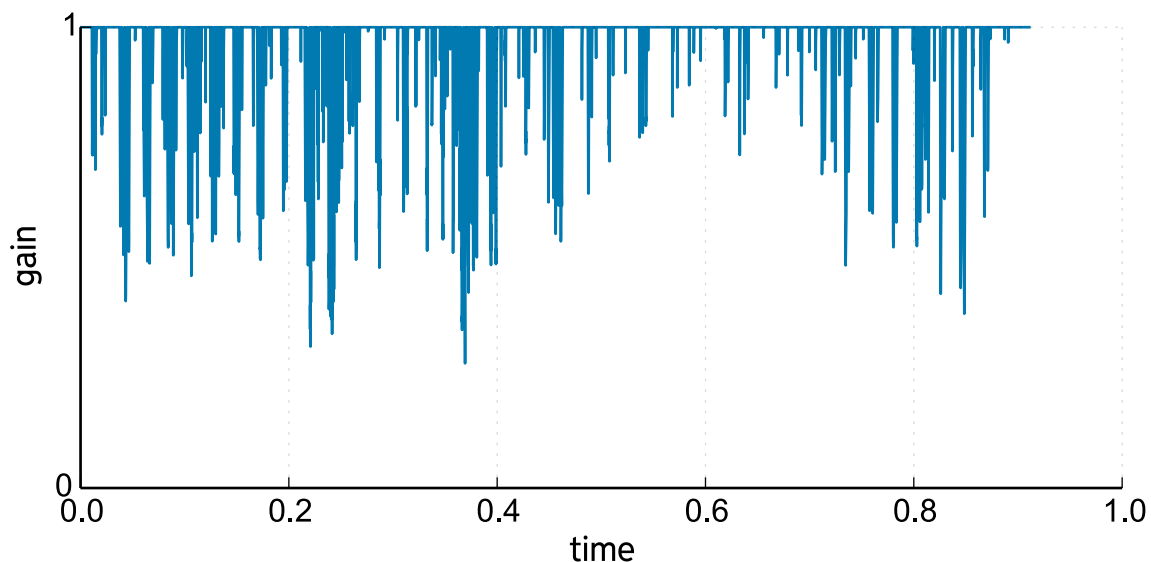
This is a job for a limiter! Let's write one.

Varying gain over time

What we need is a gain envelope which changes over time, in response to the signal. That's simple enough: for each sample, we see if it's over the limit, and reduce it as much as needed.

Example C++ code

```
struct ClipGain {  
    double limit = 0.25;  
  
    double gain(double v) {  
        // Is the sample outside our range?  
        if (std::abs(v) > limit) {  
            // Enough gain to bring it below the limit  
            return limit/std::abs(v);  
        }  
        // Otherwise, leave it alone (gain of 1)  
        return 1;  
    }  
  
    double sample(double v) {  
        return v*gain(v);  
    }  
};
```



We can see the gain levels reacting to the input signal, with the lowest values corresponding to the biggest peaks in the input. Let's have a listen:



0:00 / 0:12



Ah. You see, what we've written here is just a hard-clipper: any sample above the amplitude limit gets reduced to exactly the limit.

Smoothing out the gain changes

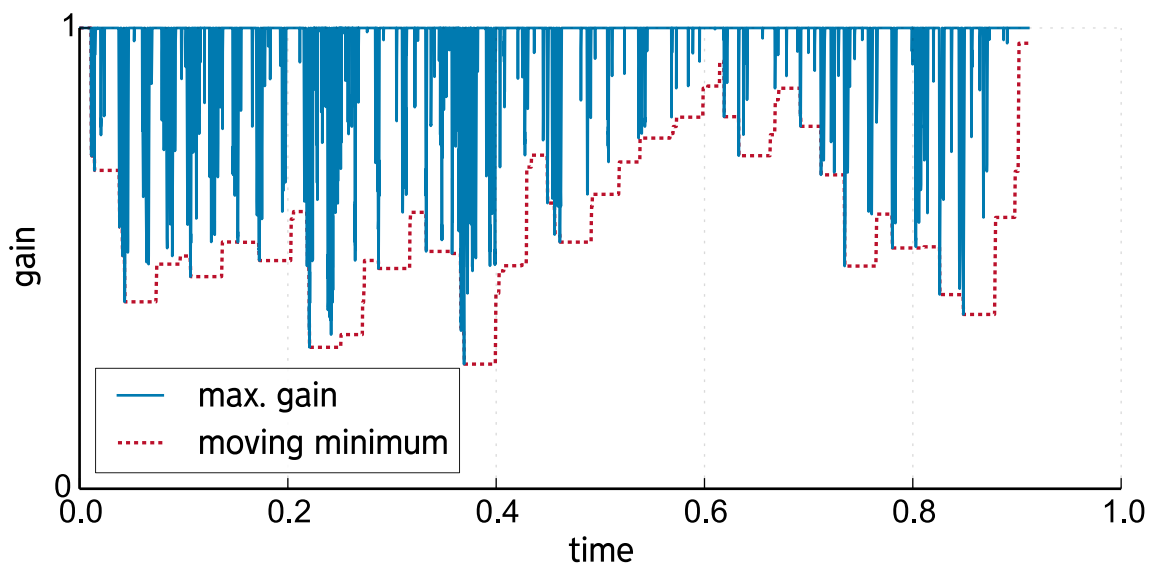
That gain value is the *maximum* gain we can apply to avoid the output going above our limit, but sticking exactly to that limit means the gain curve changes really quickly, and this distorts our signal. We can avoid this by creating a smoother gain curve, which stays strictly below the gain values we used above.

There's a neat way to do this, using two other processors we discussed recently:

Peak-hold

When I say "peak-hold" here, I'm referring to a moving maximum. This isn't the only definition, so if this causes confusion then I might have to rename some of my [DSP library](#) classes as well as my previous blog-post. 🙄

We can turn a [peak-hold processor](#) upside-down to get a moving minimum instead of a maximum. For each input sample, the next N output samples are guaranteed to be $\leq$ that input sample:



Example C++ code

```

struct MinHold {
    double limit = 0.25;
    double holdMs = 30;

    signalsmith::envelopes::PeakHold<double> peakHold{0};

    void configure(double sampleRate) {
        int holdSamples = (holdMs*0.001)*sampleRate;
        peakHold.resize(holdSamples);
    }

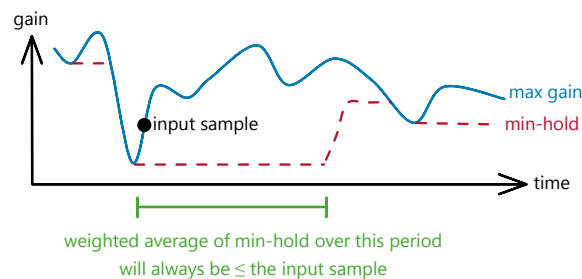
    double gain(double v) {
        double maxGain = 1;
        if (std::abs(v) > limit) {
            maxGain = limit/std::abs(v);
        }
        // To make it a minimum, negate the input and output
        return -peakHold(-maxGain);
    }

    double sample(double v) {
        return v*gain(v);
    }
};

```

Finite-length smoothing

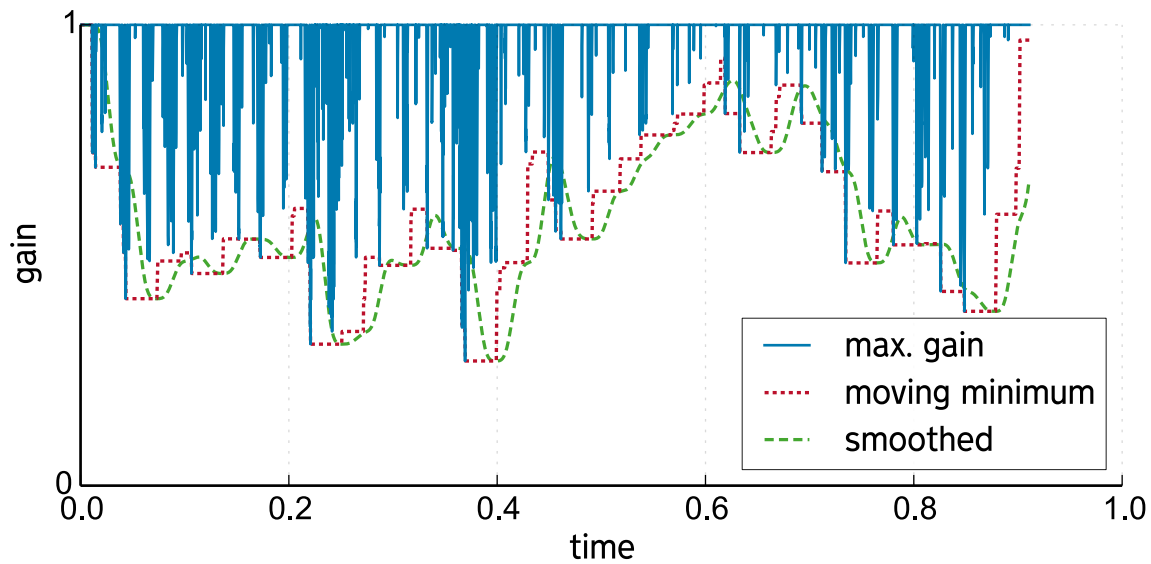
If we then take a (weighted) moving average no longer than our moving minimum, it will smooth out the result while still remaining below our input:



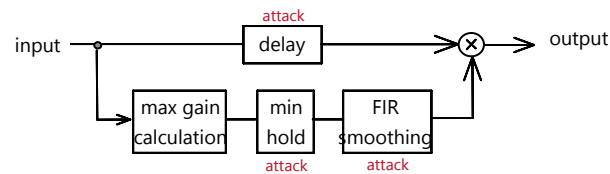
A box-filter (rectangular moving average) would work here, but we [recently looked at](#) how you can cascade box-filters to get something even better. The key point is that if the moving-average is finite, the output will be strictly below the input.

Latency

We end up with a smoothly-changing gain value that is always under the maximum gain... from N samples ago.



To get our new smooth gain curve to align properly, we just need to delay the original input signal by the same number of samples as our moving-minimum and smoothing:



We've labelled this min-hold and smoothing duration as "attack", for reasons which will make sense in a bit.

Example C++: basic limiter

```

struct BasicLimiter {
    double limit = 0.25;
    double attackMs = 30;

    // For the gain curve
    signalsmith::envelopes::PeakHold<double> peakHold{0};
    signalsmith::envelopes::BoxStackFilter<double> smoother{0};
    // For the input signal
    signalsmith::delay::Delay<double> delay;

    int attackSamples = 0;
    void configure(double sampleRate) {
        attackSamples = attackMs*0.001*sampleRate;
        peakHold.resize(attackSamples);
        smoother.resize(attackSamples, 4);
        smoother.reset(1);
        delay.resize(attackSamples + 1);
    }
    int latencySamples() {
        return attackSamples;
    }

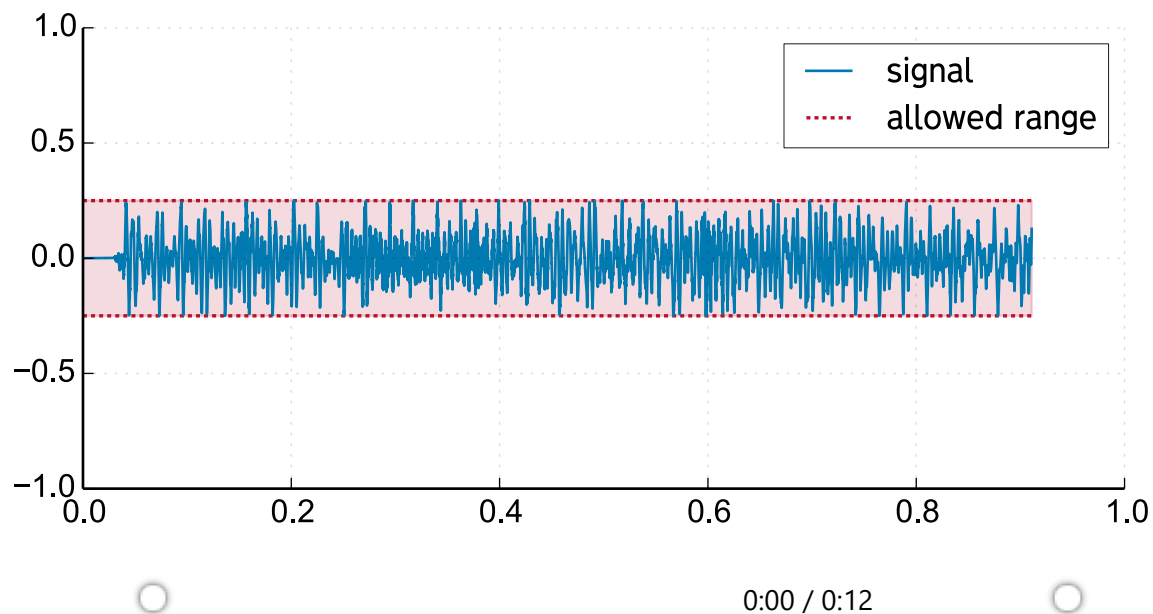
    double gain(double v) {
        double maxGain = 1;
        if (std::abs(v) > limit) {
            maxGain = limit/std::abs(v);
        }
        double movingMin = -peakHold(-maxGain);
        return smoother(movingMin);
    }

    double sample(double v) {
        double g = gain(v);
        v = delay.write(v).read(attackSamples);
        return v*g;
    }
};

```

The delay, peak-hold and FIR smoothing ([BoxStackFilter](#)) are existing implementations from the [Signalsmith DSP Library](#). We've talked about the peak-hold/smoothing algorithms before, so you can implement the same thing yourself if you want.

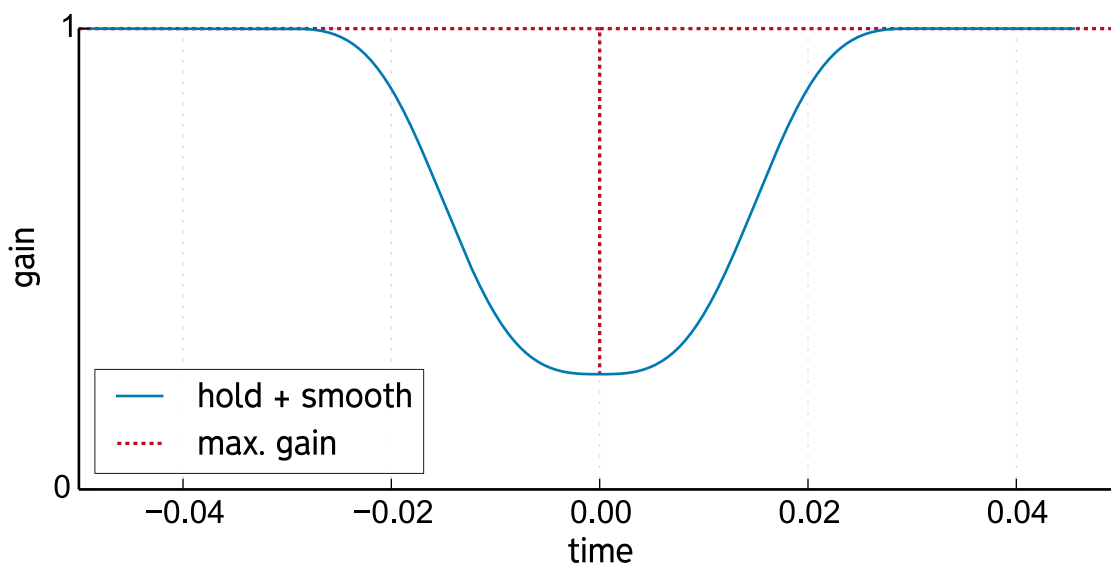
With that, here's our output:



The waveform now fits within our bounds, and sounds reasonably like [the original](#). So, this is a limiter!

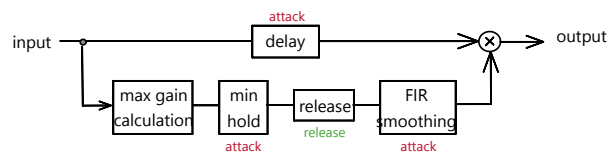
Release curve

That example was computed with a 30ms peak-hold and smoothing time - meaning the gain starts reducing 30ms before each peak (with 30ms of latency) and takes 30ms to return to settle back afterwards. Here's a graph of the output gain over time (compensated for latency):



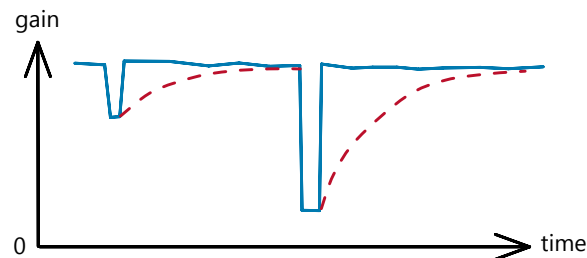
This is the output gain over time, given an input with a loud impulse at $t = 0$. The "max. gain" line shows an instantaneous drop in gain (to bring the impulse within range), which is what we are trying to avoid.

Because of how we perceive transients and volume-changes, we often want a shorter attack/anticipation than this - but still have a longer release. This is fairly simple to do - we use a shorter hold/smoothing, and put a release into the gain-curve calculation path.



If we place the release step before the smoothing, it'll also smooth out any sharp corners the release curve might have.

One of the simplest release curves is exponential. Every sample, the gain envelope either decreases to match the input, or increases (at most) a fixed percentage towards the input:



This exponential decay is simple to calculate, and takes up very little memory - we'll talk more about it later!

Example C++: exponential release

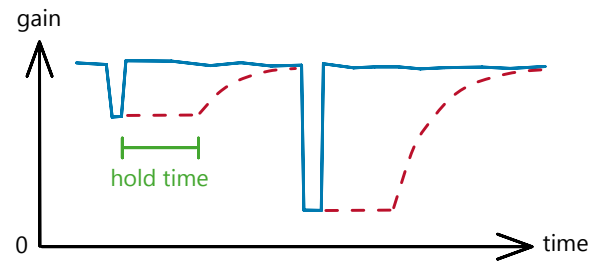
```
struct ExponentialRelease {
    double releaseSlew;
    double output = 1;

    ExponentialRelease(double releaseSamples) {
        // The exact value is `1 - exp(-1/releaseSamples)`
        // but this is a decent approximation
        releaseSlew = 1/(releaseSamples + 1);
    }

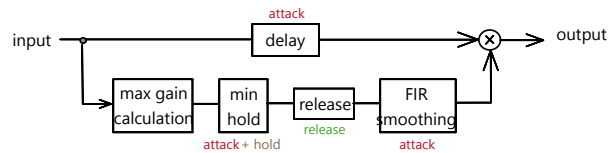
    double step(double input) {
        // Move towards input
        output += (input - output)*releaseSlew;
        output = std::min(output, input);
        return output;
    }
};
```

Extra hold time

Similarly, we might want the gain curve to hold constant for a bit, before the release curve starts. If there are lots of peaks in a row, this prevents our gain envelope from fluttering.

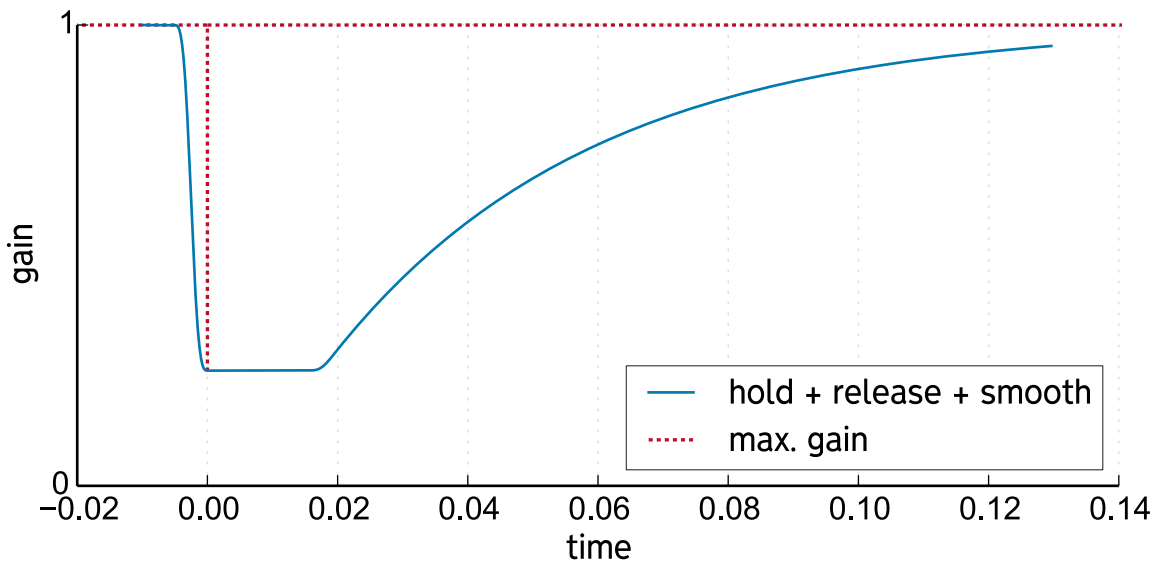


This is also pretty simple to do: we increase the time for our moving-minimum, but not the smoothing or latency. Our processing flow looks the same as before, only the lengths have changed:



Since the FIR smoothing also extends the release tail slightly, you may want to shorten the release length accordingly, but that's a judgement call.

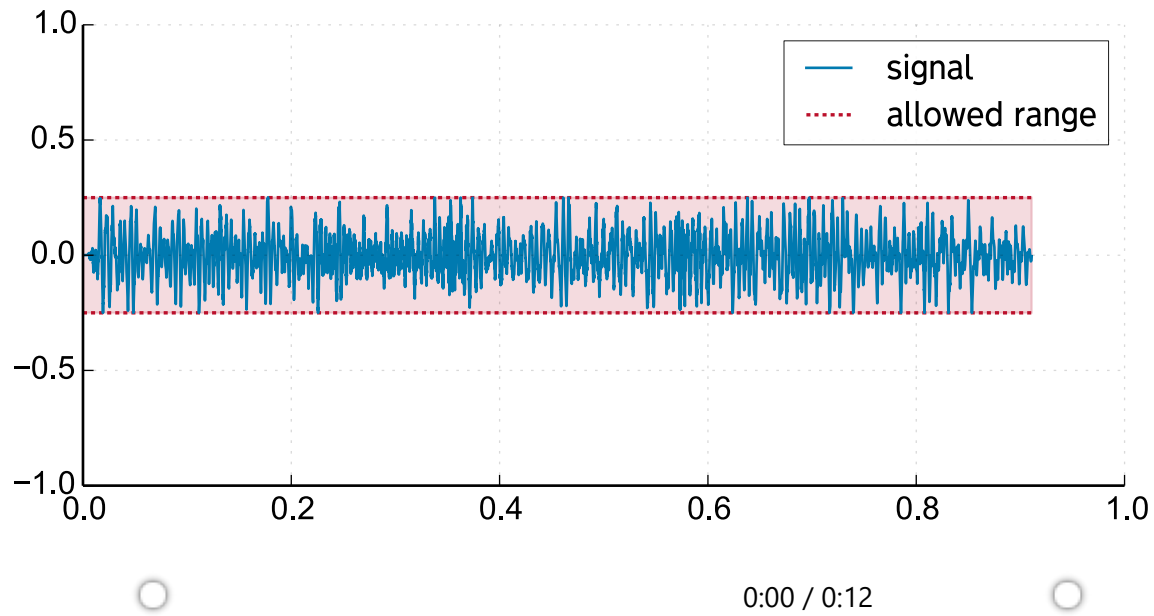
Let's look at our gain response curve for this updated flow:



You can see that the attack is shorter, and there is a distinct flat section for the hold, before the exponential release starts.

Examples

Let's try this new setup on our example sound. For this, I've used a 5ms attack, 15ms extra hold time, and a 40ms release:



And here's the implementation:

Example C++: limiter with attack/hold/release

```

struct LimiterAttackHoldRelease {
    double limit = 0.25;
    double attackMs = 5;
    double holdMs = 15;
    double releaseMs = 40;

    signalsmith::envelopes::PeakHold<double> peakHold{0};
    signalsmith::envelopes::BoxStackFilter<double> smoother{0};
    // We don't need fractional delays, so this could be nearest-sample
    signalsmith::delay::Delay<double> delay;
    ExponentialRelease release; // see the previous example code

    int attackSamples = 0;
    void configure(double sampleRate) {
        attackSamples = attackMs*0.001*sampleRate;
        int holdSamples = holdMs*0.001*sampleRate;
        double releaseSamples = releaseMs*0.001*sampleRate;
        release = ExponentialRelease(releaseSamples);

        peakHold.resize(attackSamples + holdSamples);
        smoother.resize(attackSamples, 3);
        smoother.reset(1);

        delay.resize(attackSamples + 1);
    }
    int latencySamples() {
        return attackSamples;
    }

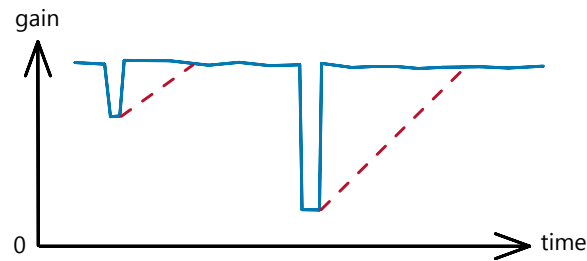
    double gain(double v) {
        double maxGain = 1;
        if (std::abs(v) > limit) {
            maxGain = limit/std::abs(v);
        }
        double movingMin = -peakHold(-maxGain);
        double releaseEnvelope = release.step(movingMin);
        return smoother(releaseEnvelope);
    }

    double sample(double v) {
        double g = gain(v);
        double delayedV = delay.write(v).read(attackSamples);
        return delayedV*g;
    }
};

```

Alternative release curves

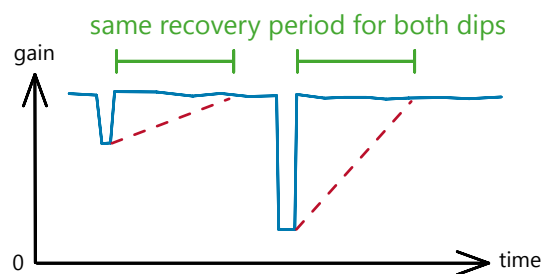
There are a *ton* of ways to implement release curves, some of which are also very simple. One example would be to limit the gain to increase at a fixed rate:



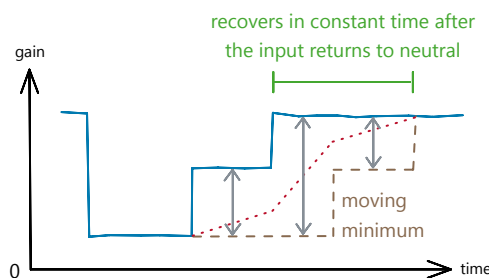
If the gain can only increase at a fixed rate, it will take longer to recover from bigger dips. This may or may not be what you want!

Constant-time release

There's another one I like, where the curve returns back to neutral in a fixed amount of time, regardless of how deep the dip is:



The algorithm here is quite simple, but figuring it out was a fun little puzzle. You start with a moving-minimum, and then your output moves with a gradient proportional to the difference between the input and that minimum:



Since this involves a peak-hold, it only really makes sense for integer release times, and there are a couple of other slight complications, but I think it's still pretty neat!

Example C++: constant-time release

```
struct ConstantTimeRelease {
    signalsmith::envelopes::PeakHold<double> peakHold{0};
    double gradientFactor = 1;
    double output = 1;

    ConstantTimeRelease(int releaseSamples) {
        releaseSamples = std::max(releaseSamples, 1);
        // This will finish its release 0.01 samples too early
        // but that avoids numerical errors
        gradientFactor = 1.0/(releaseSamples - 0.01);
        peakHold.resize(releaseSamples);
    }
};
```

```

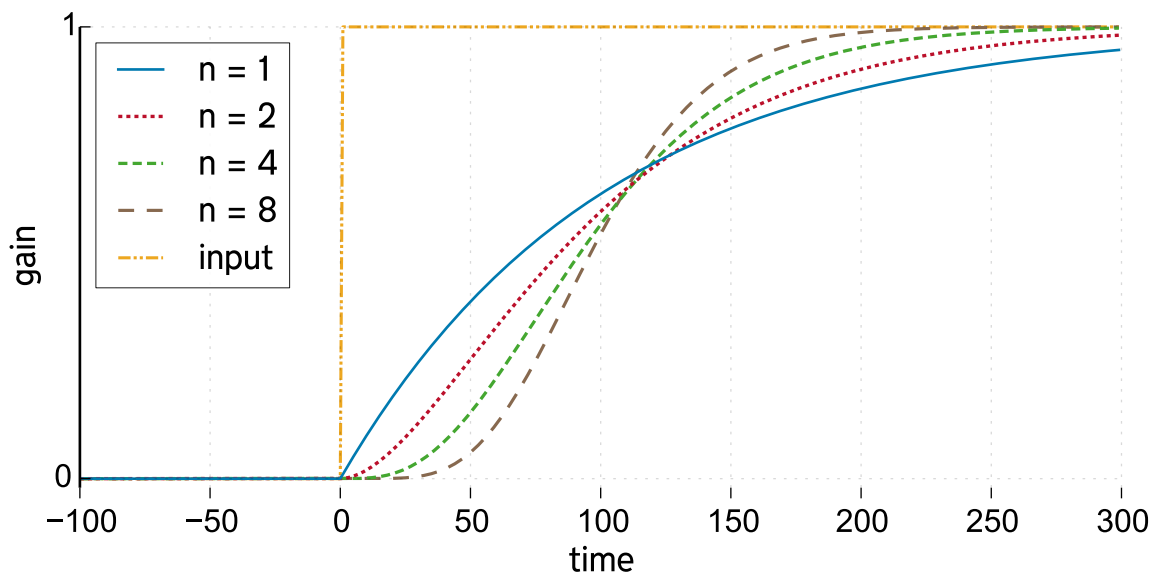
    peakHold.reset(1); // start with gain 1
}

double step(double input) {
    // We need the peak from one sample back
    double prevMin = -peakHold.read();
    peakHold(-input);
    // Gradient is proportional to the difference
    output += (input - prevMin)*gradientFactor;
    output = std::min(output, input);
    return output;
}
};

```

Cascaded releases

Both the exponential-release and the constant-time shown above can be cascaded in series to produce a smoother curve. Here's what happens when you cascade n exponential release processors:



This shows each cascade's **step response**: what happens when the input changes from 0 to 1. Each cascade is made from n exponential releases of length $100/n$, for easier comparison of the shapes.

Closed-form expression for the cascaded-exponential release curve

Above, we scaled the release lengths by $1/n$, but you might have other ideas - for example, specifying how long the step response should take to rise by 95% of the step size.

For that, it's useful to have a closed-form expression for the release curve, at least for the continuous-time equivalent. We'll normalise the release time, such that the impulse response of our smoother is the one-sided exponential:

$$f(x) = H(t) \exp(-t)$$

where $H(x)$ is the [Heaviside step function](#).

A cascade of M such releases will have a step-response equal to $H(x)$ convolved repeatedly

with $f(x)$. We can write this as a sequence of functions:

$$r_0 = H$$

$$r_{M+1} = r_M * f$$

This has a fairly neat closed-form solution:

$$r_M(t) = H(t) \left(1 - \exp(-t) \sum_{m=0}^{M-1} \frac{t^m}{m!} \right)$$

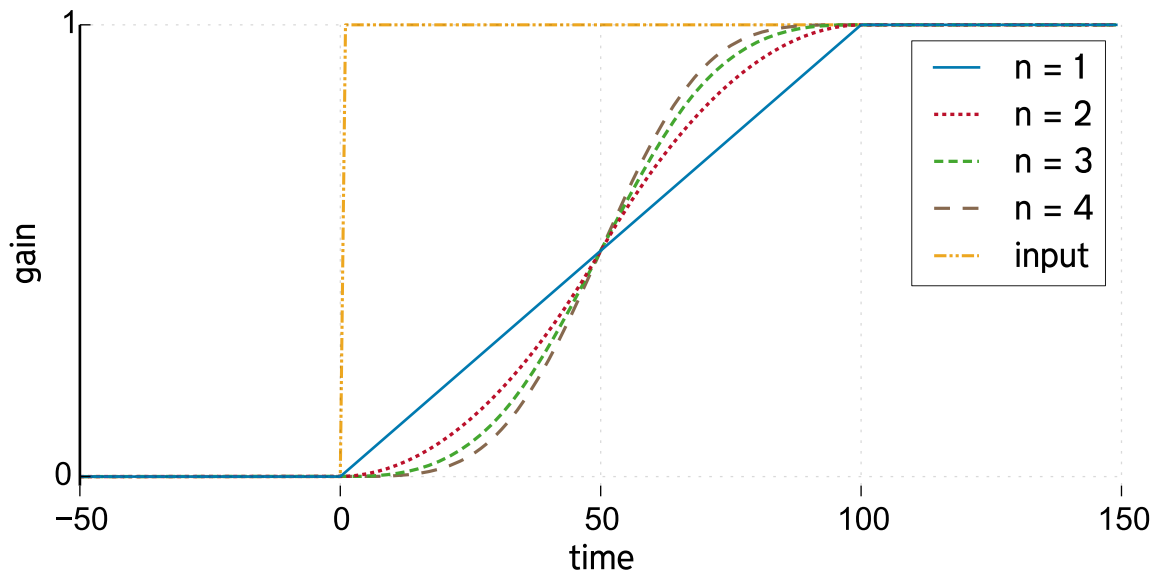
I don't have a nice inverse for this, but it increases monotonically so you can find a solution iteratively, for a given depth M and threshold (e.g. 0.95).

This is trivially true for $M = 0$ (where the $\sum$ just disappears), and we can prove it for $M > 0$ by induction:

$$\begin{aligned} r_{M+1}(t) &= \int_{-\infty}^{\infty} f(t - \tau) r_M(\tau) d\tau \\ &= \int_0^t \exp(\tau - t) \left(1 - \exp(-\tau) \sum_{m=0}^{M-1} \frac{\tau^m}{m!} \right) d\tau \\ &= \int_0^t \exp(\tau - t) d\tau - \exp(-t) \int_0^t \sum_{m=0}^{M-1} \frac{\tau^m}{m!} d\tau \\ &= (1 - \exp(-t)) - \exp(-t) \sum_{m=0}^M \frac{\tau^{m+1}}{(m+1)!} \\ &= 1 - \exp(-t) \sum_{m=0}^M \frac{\tau^m}{m!} \end{aligned}$$

Each extra stage adds another level of continuity - so one release-stage produces continuous values, two will produce continuous gradient, and so on.

You can cascade constant-time releases in the same way:



We've used the same $100/n$ release-time scaling as before, but constant-time releases require integer lengths, so for $n = 3$ we used $[33, 33, 34]$.

For both of these, when the cascade has more stages, the release curve has a much slower start (starting to look similar to the hold period) while the main part of the release is steeper.

... and even more choices

Even starting from this fairly simple design, we've had to make some decisions about the shape of our release curve, and the shape of the FIR smoothing for the attack.

These choices will affect the particular sound of a limiter - and here are a few more things to consider:

Inter-sample peaks

So far, we've only considered the values of individual samples. But when a signal is re-sampled, or converted to analogue (to head to the speakers!), it can produce values outside our allowed range.

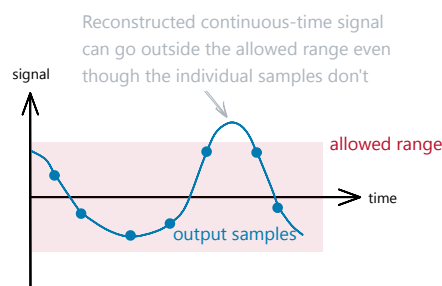
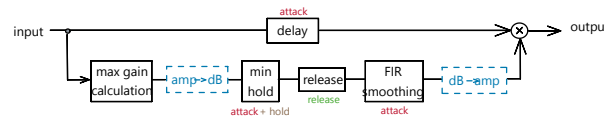


Diagram showing inter-sample peaks when reconstructing a signal from samples. The exact location and values of these peaks will depend on the reconstruction/resampling method.

These are called "inter-sample peaks", and a good limiter should make sure these are contained as well. The most straightforward approach would be to actually upsample the signal, see what values come out, and then use these in the maximum-gain calculation.

Loudness scale

Our design (and example code) above dealt directly with gain multipliers. But we could easily adapt it to dB instead:



The same basic design, but with some extra blocks (**dashed border**) in the gain path for converting to/from dB.

We convert our "maximum gain" into dB (or any other scale we want!), perform all the same smoothing and release logic as before, and then convert back afterwards. A lot of compressors work on a dB scale - should our limiter use dB as well?

Latency and imperfect anticipation

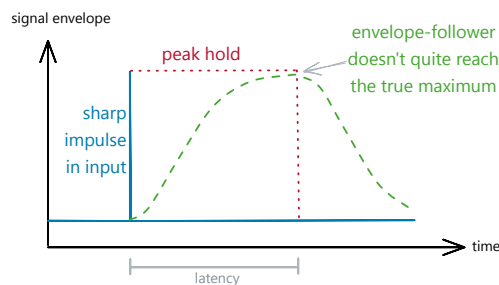
Our design uses a delay and FIR smoothing to anticipate peaks. But do we need to *perfectly* anticipate everything?

If we were prepared to accept a bit of clipping on sudden peaks, our smoothers could be longer than the peak-hold length. We'd have to hard-clip the output, but this would let us use IIR smoothers, or could be used trade distortion for latency (by reducing the delay and peak-hold durations). In a live situation where 10ms of latency would be a deal-breaker, it might be worth it.

I had some great discussions with [Dario Sanfilippo](#) about this.

He's presenting a design at [IFC-22](#) which uses an envelope follower made from a more approximate peak-hold, and a cascade of 1-pole filters instead of an FIR (which act similarly to the [exponential releases](#) described above).

These peak-hold/smoothers alternatives use very little memory - but these IIR smoothers never quite reach their target, so the envelope-follower can undershoot:



This diagram is the other way up to ours, because it's trying to create a smooth envelope that follows the signal's peaks. This is then inverted to create a smooth gain envelope.

If you configure it appropriately, this overshoot is small (and short) enough to not be a problem, so you can just hard-clip the output with only minimal distortion.

Envelope Following Via Cascaded Exponential Smoothers for Low-Distortion Peak Limiting and Maximisation

Dario Sanfillipo

Proceedings of the International Faust Conference (IFC-22)

Adaptive release

Does the release time have to be constant? Are there things we can detect in the input to determine when we should recover quicker or slower?

Mixed-size release cascades

The release cascades we looked at were using the same length for all of their stages, but we can use any combination of sizes. Using smaller releases in combination with bigger ones can soften the corners without disrupting the overall shape - and we could even mix exponential/constant-time stages.

Multi-band processing

Multi-band compressors are pretty common, where you perform a band-split and then compress each band separately.

Similarly, having a limiter which reduces some bands more than others (or has different attack/hold/release times) is an appealing idea. However, if we distribute the gain-reduction differently between the bands, they might add up differently in the attack/hold/release stages. This could be solved by iterating, or just having a second full-band limiter afterwards, but it's less straightforward than for compressors.

We listed a bunch of options, some of which are clear improvements, but some are just ... design decisions. You can have a play around, and if there are choices which feel "right" to you, go for it!

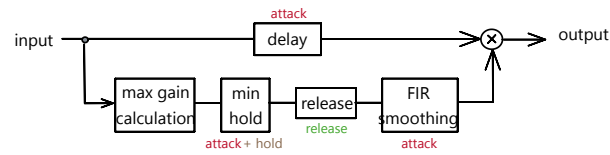
I personally don't like when there are too many alternatives like this. If there isn't a choice that seems natural and obvious, then it becomes a judgement call - and how do you know if your judgement is good?

At this point, my usual move is to make up some metrics for things I want, and use them to explore the trade-offs of different combinations. I'm not going to get into that here, though - it's probably a whole article by itself, which I hope to get to at some point.

Conclusion

While there are ways it could be extended and improved, we've gone through a simple design for a limiter, where we specify:

- maximum output value (limiter threshold)
- attack time
- hold time
- release time



This is a "lookahead" limiter, meaning it uses latency to anticipate peaks, and react slightly ahead of them.

I hope it can be a useful starting-point for anyone who hasn't considered writing one before.

— Geraint

Comments/questions? Get in touch by [email](#) / [Twitter](#), or join the [mailing-list](#) for blog updates.